Package 'ssGSEA3'

October 23, 2026

Title Single-Sample and Site-Centric Gene Set Enrichment Analysis (ssGSEA/PTM-SEA)

Version 1.0.0

Description Tools for gene-centric (ssGSEA) and site-centric (PTM-SEA)
single-sample gene set enrichment analysis: GCT preprocessing into
gene-centric or single-site-centric reports, a feature-level (gene- or
site-level) bootstrap enrichment/significance engine, and summary heatmap reporting with
HALLMARK and PTMSigDB term groupings.

License file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.1)

Imports cmapR,
ComplexHeatmap,
circlize,
magrittr,
glue,
dplyr,
doParallel,
foreach,
gtools,
verification,
methods,
stats,
parallel,
graphics,
grDevices

Suggests testthat (>= 3.0.0),
withr,
optparse,
yaml,
AnnotationDbi,
org.Hs.eg.db,
org.Mm.eg.db,
org.Rn.eg.db,
org.Dr.eg.db,
knitr,
rmarkdown

Config/testthat/edition 3

VignetteBuilder knitr

LazyData true

Contents

hallmark_categories 2
hallmark_category_colors 3
plot_rank_diagnostics 3
plot_ssgsea_heatmap 4
preprocessGCT 6
read_gene_sets 8
RefSeq2UniProt 9
select_top_terms 10
ssGSEA3 10

Index **13**

`hallmark_categories` *HALLMARK gene set process categories*

Description

Named character vector mapping each of the 50 MSigDB HALLMARK gene set names to a broad process category (signaling, immune, metabolic, etc.), used to color-code and group rows in summary heatmaps.

Usage

```
hallmark_categories
```

Format

A named character vector of length 50; names are HALLMARK gene set IDs, values are one of "signaling", "immune", "development", "proliferation", "cellular component", "metabolic", "DNA damage", or "pathway".

Source

Broad Institute MSigDB HALLMARK collection; category assignments originate from the lab's ssGSEA reporting scripts (`pw_hm()`).

hallmark_category_colors *Color palette for HALLMARK process categories*

Description

Color palette for HALLMARK process categories

Usage

```
hallmark_category_colors
```

Format

A named character vector of hex color codes, one per category value in hallmark_categories.

plot_rank_diagnostics *Rank ("mountain") plot diagnostic for ssGSEA/PTM-SEA results*

Description

For each selected gene set, draws one page with one panel per sample: the sample's original expression values, ranked high to low, with the gene set's members marked along that ranking. This is the classic GSEA enrichment-plot diagnostic, adapted to ssGSEA/PTM-SEA – useful for visually confirming that a significant score is actually driven by members clustered toward one end of the ranking, not a fluke.

Usage

```
plot_rank_diagnostics(
  result,
  terms = NULL,
  top_n = 10,
  direction = c("both", "pos", "neg"),
  fdr.max = 0.05,
  samples = NULL,
  file = "rank-plots.pdf",
  mfrow = c(3, 3)
)
```

Arguments

result	An ssgsea_result, as returned by ssGSEA3().
terms	Character vector of gene set names to plot. If NULL (default), gene sets are chosen via select_top_terms() using top_n/ direction/fdr.max.
top_n, direction, fdr.max	Passed to select_top_terms() when terms is NULL.
samples	Character vector of sample names to plot (as panels on each gene set's page). Defaults to all samples in result.

file	Path to write a (possibly multi-page) PDF to. If NULL, plots to the current graphics device instead (one page per term).
mfrow	Panel layout (rows, cols) per page; samples beyond prod(mfrow) for a single gene set spill onto additional pages.

Details

Unlike the original implementation (which re-parsed the written score/ p-value/FDR GCT files back off disk for every plot), this draws directly from the result object already in memory.

Value

(Invisibly) the character vector of gene sets plotted.

---

plot_ssgsea_heatmap *ssGSEA/PTM-SEA summary heatmap*

Description

Reworks the lab's existing pw_hm() heatmap (base pheatmap) onto ComplexHeatmap. Selects gene sets via select_top_terms() by default (FDR-significant, then top-N by direction), colors a symmetric blue-white- red scale centered at zero, and marks FDR-significant cells with *.

Usage

```
plot_ssgsea_heatmap(
  result,
  terms = NULL,
  top_n = 10,
  direction = c("both", "pos", "neg"),
  fdr.max = 0.05,
  samples = NULL,
  split_by_db_category = TRUE,
  custom_categories = NULL,
  cluster_rows = TRUE,
  cell.size = 10,
  title = NULL,
  nes.range = NULL,
  title_fontsize = 14,
  show_row_names = TRUE,
  show_category = TRUE,
  show_heatmap_legend = TRUE,
  show_row_dend = TRUE
)
```

Arguments

result	An ssgsea_result, as returned by ssGSEA3().
terms	Character vector of gene set names to plot. If NULL (default), gene sets are chosen via select_top_terms() using top_n/ direction/fdr.max.

`top_n`, `direction`, `fdr.max`
: Passed to `select_top_terms()` when `terms` is `NULL`; `fdr.max` is also the threshold used for marking significant cells with `*`.

`samples`
: Character vector of sample names (columns) to include. Defaults to all samples in `result`.

`split_by_db_category`
: If `TRUE` (default), auto-detect and split by a shared `CATEGORY-name` prefix (e.g. PTMSigDB) when HALLMARK grouping doesn't apply.

`custom_categories`
: Optional row grouping for databases with no HALLMARK/PTMsigDB naming convention to auto-detect (e.g. REACTOME) – either a path to a 2-column file mapping gene set name to category (one pair per line, tab- or comma-delimited; a header row is harmless, since it just becomes an entry that won't match any real gene set name and is silently unused), or an already-built named character vector (names = gene set name, values = category label). Terms present in `terms` but absent from this mapping are grouped into their own `""`-labeled category (colored grey) rather than erroring, since a custom category file will often not cover every gene set in a database. Colors are auto-assigned from a qualitative palette (`grDevices::hcl.colors()`, `"Dark 3"`) since, unlike HALLMARK, there's no fixed bundled color table for arbitrary categories. Takes priority over HALLMARK/ `split_by_db_category` auto-detection when supplied.

`cluster_rows`
: Controls row order/dendrogram; forwarded as-is to `ComplexHeatmap::Heatmap()`'s own `cluster_rows`, which accepts three different kinds of value:

  - `TRUE` (default): cluster fresh from the matrix being plotted. Within each group/slice when HALLMARK or prefix-based grouping applies, across all rows otherwise. Dendrogram lines are drawn.
  - `FALSE`: no clustering at all – rows are shown in exactly the order given in `terms` (or the order `select_top_terms()` returned).
  - A precomputed `hclust`/`dendrogram` object, or a function that takes the matrix being plotted and returns one: your clustering is used exactly as given, *without* being re-sorted to better fit this particular matrix's values (`Heatmap()`'s own `row_dend_reorder` otherwise does this by default, which is why this case turns it off automatically here). Useful for drawing several `plot_ssgsea_heatmap()` results side by side (e.g. comparing two pipeline versions) with identical row order/dendrogram across panels, even though each panel's underlying matrix differs.

`cell.size`
: Fixed cell width/height, in points, matching `pw_hm()`'s own `cw`/`ch` default of 10 – keeps cells square regardless of the number of rows/columns, rather than stretching to fill the plotting device (which is what a bare `ComplexHeatmap::Heatmap()` call does by default when `width`/`height` aren't set).

`title`
: Optional heatmap title.

`nes.range`
: Optional `c(min, max)` to fix the color scale instead of auto-computing it from the terms/samples actually being plotted. Pin this to the same values across several heatmaps – e.g. one showing all significant terms and another showing a hand-picked illustrative subset – so a subset with modest scores reads as visually modest rather than stretching to fill the color scale's extremes on its own narrower range. When computing this externally from a larger pool (e.g. a whole gene set database), do NOT use that pool's literal min/max – a single extreme outlier gene set (common in real data; e.g. a tiny gene set with an inflated score) will dominate the range and wash out every other heatmap pinned

to it, including ones with genuinely large, real effect sizes. Use a percentile clamp instead (e.g. the 1st/99th percentile of NES among gene sets significant somewhere in that pool) so a handful of outliers can't blow out the shared scale.

`title_fontsize` Font size for `title`. Default 14 suits a standalone heatmap with several sample columns; narrow panels (e.g. 2 columns) may need a smaller size to avoid the title overflowing its own column width, particularly when concatenating several calls' results side by side.

`show_row_names`, `show_category`, `show_heatmap_legend`, `show_row_dend`
Individually toggle the row names, category color bar/title, color-scale legend, and row dendrogram – all default `TRUE` for standalone use, but useful to selectively disable when concatenating several calls' results into one multi-panel comparison figure (e.g. category bar only on the first panel, row names only on the last, one shared legend).

Details

Row grouping is determined in this order:

- If `custom_categories` is supplied, rows are split/colored by that mapping (see below) – takes priority over the two auto-detected modes.
- Otherwise, if any term is a bundled hallmark_categories name, rows are split and colored by HALLMARK process category (matching `pw_hm()`'s behavior exactly), with the `HALLMARK_` prefix stripped from row labels.
- Otherwise, if `split_by_db_category` and the terms carry a shared `CATEGORY-name` prefix convention (as in PTMSigDB: `KINASE-`, `PERT-`, `PATH-`, `DISEASE-`) with more than one distinct prefix present, rows are split by that prefix.

In every case, rows are hierarchically clustered by default (`cluster_rows = TRUE`) using `ComplexHeatmap`'s own built-in clustering rather than `pw_hm()`'s `seriation::seriate()` – avoids an extra dependency for what's typically a very similar row order. When a grouping applies, clustering happens *within* each group/slice (slice order itself is left as the groups' first-encountered order, not reclustered). See the `cluster_rows` argument below for the two other values it accepts besides `TRUE/FALSE`.

Value

A ComplexHeatmap::Heatmap object – draw it with `ComplexHeatmap::draw()`, or save with e.g. `pdf()/png()` around that call.

`preprocessGCT` *Preprocess a GCT file for ssGSEA or PTM-SEA*

Description

Converts a raw GCT file into one of three report levels:

- `"gc"` (gene-centric): collapses multiple rows per gene into one, per `mode`. **This is the level `ssGSEA3()` expects** – exactly one row per gene, matching its assumption that each gene contributes a single ranked value to the enrichment statistic.

- `"ssc"` (single-site-centric): the PTM-SEA path. Explodes multiply-modified peptides into one row per site and builds a PTM-SEA-compatible site ID – also one row per (site-level) scoring unit, so also directly usable by `ssGSEA3()`.
- `"gcr"` (gene-centric-redundant): relabels rows with gene symbols but does *not* collapse – every original row is kept (deduplicating names only, e.g. `GeneA`, `GeneA_1`). This is intended as an inspection/reporting format (e.g. seeing every individual peptide/probe measurement under its gene symbol before choosing how to collapse it), **not** as `ssGSEA3()` input: feeding it multiple rows for the same gene in would each count as an independent "hit" in the enrichment ranking, giving genes with more rows disproportionate influence on the score for reasons unrelated to real biology. Use `"gc"` for ssGSEA scoring instead.

Usage

```
preprocessGCT(
  gct.str,
  level = c("ssc", "gc", "gcr"),
  id.type = c("sm", "ph"),
  id.type.out = c("uniprot", "refseq", "seqwin"),
  acc.type = c("uniprot", "refseq", "symbol"),
  organism = NULL,
  seqwin.col = "VMsiteFlanks",
  gene.col = "geneSymbol",
  fix.gene.names = FALSE,
  fix.gene.names.column = "id.description",
  fix.gene.names.regexpr = ".* GN\\=(.*)$",
  humanize.gene.names = FALSE,
  loc = TRUE,
  mode = c("mean", "median", "sd", "SGT", "abs.max"),
  SGT.col = "subgroupNum",
  mod.res = c("S|T|Y", "K"),
  mod.type = c("p", "ac", "ub"),
  appenddim = TRUE
)
```

Arguments

`gct.str`	Path to the input GCT (v1.2/v1.3) file.
`level`	One of `"ssc"`, `"gc"`, `"gcr"` (see above). Use `"gc"` or `"ssc"` for data that will be scored with `ssGSEA3()`; `"gcr"` is not recommended for that purpose (see above).
`id.type`	Input site-ID notation: `"sm"` (Spectrum Mill) or `"ph"` (Philosopher). Only used when `level = "ssc"`.
`id.type.out`	Output site-ID type: `"uniprot"`, `"refseq"`, or `"seqwin"`. Only used when `level = "ssc"`.
`acc.type`	Accession type of the `rid` in the input GCT: `"uniprot"`, `"refseq"`, or `"symbol"`. Only used when `level = "ssc"`.
`organism`	One of `"human"`, `"mouse"`, `"rat"`, `"zebrafish"`. Required only when mapping to UniProt (`level = "ssc"`, `id.type.out = "uniprot"`, `acc.type` in `c("refseq", "symbol")`).
`seqwin.col`	Column containing `|`-separated flanking sequences; only used when `id.type.out = "seqwin"`.

gene.col	Column listing gene symbols; used for gene-centric levels and by fix.gene.names.
fix.gene.names	If TRUE, replace gene.col with gene names extracted from fix.gene.names.column via fix.gene.names.regexpr – a workaround for gene symbols corrupted by spreadsheet date-autoformatting (e.g. "SEPT10" becoming a date).
fix.gene.names.column	Column to extract corrected gene names from.
fix.gene.names.regexpr	Regular expression identifying the gene name within fix.gene.names.column.
humanize.gene.names	If TRUE, upper-case gene symbols (for mouse or rat data). Only used when level = "gc".
loc	If TRUE (recommended), only fully localized PTM sites are kept. Only used when level = "ssc".
mode	How multiple rows per gene are combined when level = "gc", and how multiply-detected duplicate sites are combined when level = "ssc" (only "mean", "median", "abs.max" apply to "ssc"): "mean", "median", "sd" (keep the single row with the highest cross-sample standard deviation – i.e. the most variable/informative probe for that gene; "gc" only), "SGT" (subgroup-top: first Spectrum Mill subgroup per protein group; "gc" only), or "abs.max" (per sample, the most extreme value – for signed, log-transformed p-values).
SGT.col	Column used to collapse to subgroup-top (mode = "SGT").
mod.res	Modified residue(s), e.g. "S|T|Y" or "K".
mod.type	Modification type, e.g. "p" or "ac".
appenddim	See cmapR::write_gct().

Value

The processed cmapR::GCT object (so it can be passed straight to ssGSEA3() without re-reading the file just written) – the actual path written to disk is attached as attr(result, "path").

---

read_gene_sets *Import and merge one or more gene set databases*

Description

Reads one or more GMT files, filters each by set size, and merges them into a single collection. If any gene set member carries a PTMsigDB ;u/;d directionality suffix, the whole collection is treated as PTMsigDB-style: directionality is split out into gs.direction and stripped from members.

Usage

```
read_gene_sets(gene.set.databases, thres.min = 2, thres.max = 2000)
```

Arguments

gene.set.databases	Character vector of paths to GMT files.
thres.min	Minimum gene set size to keep.
thres.max	Maximum gene set size to keep.

Value

A list with gs, gs.names, gs.descs, size.G, and gs.direction (NULL for ordinary, non-directional gene sets).

RefSeq2UniProt *Map RefSeq accessions or gene symbols to UniProt accessions*

Description

Uses the appropriate Bioconductor org.*.eg.db annotation package for the given organism to map identifiers to UniProt accessions.

Usage

```
RefSeq2UniProt(
  ids,
  keytype = c("REFSEQ", "SYMBOL"),
  organism = c("human", "mouse", "rat", "zebrafish")
)
```

Arguments

ids	Character vector of accessions to map (RefSeq or gene symbols, per keytype). May contain version suffixes/site annotations after a . or ;, which are stripped before querying.
keytype	One of "REFSEQ" or "SYMBOL", matching the type of ids.
organism	One of "human", "mouse", "rat", "zebrafish".

Details

This replaces the previous auto-detection behavior (which sampled a few IDs and test-queried each of the human/mouse/rat/zebrafish annotation packages in turn to guess the organism). Auto-detection meant every call required all four org.*.eg.db packages to be installed, and could silently guess wrong on a small or ambiguous ID sample. Callers must now state the organism explicitly, and only the one relevant annotation package needs to be installed.

Value

A list with elements keytype, organism, and id.map (a data.frame with columns id, id.query, id.mapped, id.concat). Entries with no match have id.mapped = NA.

select_top_terms *Select top-N FDR-significant gene sets by direction*

Description

Picks gene sets that reach `fdr.max` in at least one sample, then keeps the `top_n` largest by absolute score (its most extreme per-sample value), separately for positive- and negative-scoring gene sets.

Usage

```
select_top_terms(
  result,
  top_n = 10,
  direction = c("both", "pos", "neg"),
  fdr.max = 0.05
)
```

Arguments

`result`	An `ssgsea_result`, as returned by `ssGSEA3()`.
`top_n`	Number of gene sets to keep per direction.
`direction`	`"both"`, `"pos"`, or `"neg"`.
`fdr.max`	Maximum (best-case, across samples) FDR-adjusted p-value for a gene set to be eligible.

Value

Character vector of selected gene set names.

ssGSEA3 *Single-sample GSEA / PTM-SEA enrichment, with bootstrap significance*

Description

Scores every sample in `input.ds` against every gene set in `gene.set.databases`, using a per-gene-set running-sum enrichment statistic (Subramanian et al. 2005; Abazeed et al. 2013) and, for `output.score.type = "NES"`, a feature-level (gene- or site-level) bootstrap to obtain a normalized enrichment score and p-value (see `bootstrap_es_pvalue()` for why this replaced the original permutation-based approach).

Usage

```
ssGSEA3(
  input.ds,
  output.prefix,
  gene.set.databases,
  sample.norm.type = c("rank", "log", "log.rank", "none"),
  weight = 0,
```

```
  statistic = c("area.under.RES", "Kolmogorov-Smirnov"),
  output.score.type = c("NES", "ES"),
  nperm = 1000,
  min.overlap = 10,
  correl.type = c("rank", "z.score", "symm.rank"),
  global.fdr = FALSE,
  extended.output = TRUE,
  par = FALSE,
  spare.cores = 1,
  export.signat.gct = TRUE,
  param.file = TRUE,
  seed = NULL,
  verbose = TRUE,
  log.file = "run.log"
)
```

Arguments

input.ds	Path to a GCT file (first column: gene symbols or site ids), or an already-parsed cmapR::GCT object – e.g. the result of preprocessGCT(), to avoid re-reading it from disk.
output.prefix	File prefix for output GCT files.
gene.set.databases	Character vector of paths to GMT gene set database files.
sample.norm.type	See normalize_samples().
weight	When 0, all genes are weighted equally; when > 0, actual (normalized) expression values are used and can shift the score – see correl.type.
statistic	"area.under.RES" or "Kolmogorov-Smirnov", see gsea_score().
output.score.type	"NES" (bootstrap normalized enrichment score and p-value) or "ES" (raw enrichment score only, no bootstrapping, p.value is always 1).
nperm	Number of feature-level bootstrap replicates (ignored when output.score.type = "ES").
min.overlap	Minimum number of a gene set's members that must be present in the data for it to be scored.
correl.type	"rank", "z.score", or "symm.rank" – only relevant when weight > 0, see gsea_score().
global.fdr	If TRUE, FDR is computed once across all samples' pooled p-values; otherwise per sample.
extended.output	If TRUE, include per-signature overlap statistics as additional row annotations in the output GCTs.
par	If TRUE, score gene sets in parallel across parallel::detectCores() - spare.cores workers.
spare.cores	Number of CPU cores to leave idle when par = TRUE.
export.signat.gct	If TRUE, also write one GCT per signature (to a signature_gct/ subdirectory) containing that signature's members' original expression values.

param.file	If TRUE, write a <output.prefix>_parameters.txt record of the parameters used.
seed	Optional random seed, for reproducible bootstrap resampling.
verbose	If FALSE, suppress progress messages (the log file, if log.file is not NULL, is still written).
log.file	Path to a log file recording basic progress/timing; NULL to skip logging entirely.

Value

An object of class ssgsea_result (see build_ssgsea_result()); the same information is also written to <output.prefix>-scores.gct, -pvalues.gct, -fdr-pvalues.gct, and -combined.gct (plus, if export.signat.gct, one GCT per signature).

Fixed limitation (directional/PTMsigDB gene sets)

earlier versions of this port could desync a directional (PTM-SEA/PTMsigDB ;u/;d) gene set's position/direction bookkeeping inside gsea_score() whenever a bootstrap replicate happened to resample only *some* of a gene set's members – sometimes erroring out, sometimes silently contributing a wrong enrichment score to that replicate's bootstrap null. This is now fixed via .align_geneset_to_direction(), which keeps the two in lockstep regardless of which members a given replicate happens to include. The vulnerable construction traces back to code essentially unchanged since directionality support was added to the original ssGSEA2.0 in 2016, but it could never actually manifest there: ssGSEA2.0's gene-label *permutation* reorders every gene without ever dropping one, so its (fixed, computed-once) position/direction vectors never desync. It's specifically ssGSEA3.0's feature-level bootstrap resampling *with replacement* – which can and does drop individual genes from a given replicate – that exposes it; this is a real consequence of the bootstrap rework, not a latent defect ssGSEA2.0 use was ever exposed to.

Index

∗ **datasets**
    hallmark_categories, 2
    hallmark_category_colors, 3
.align_geneset_to_direction(), *12*

bootstrap_es_pvalue(), *10*
build_ssgsea_result(), *12*

cmapR::GCT, *8*, *11*
cmapR::write_gct(), *8*
ComplexHeatmap::Heatmap, *6*
ComplexHeatmap::Heatmap(), *5*

grDevices::hcl.colors(), *5*
gsea_score(), *11, 12*

hallmark_categories, 2, *3*, *6*
hallmark_category_colors, 3

normalize_samples(), *11*

plot_rank_diagnostics, 3
plot_ssgsea_heatmap, 4
preprocessGCT, 6
preprocessGCT(), *11*

read_gene_sets, 8
RefSeq2UniProt, 9

select_top_terms, 10
select_top_terms(), *3–5*
ssGSEA3, 10
ssGSEA3(), *3, 4*, *6–8*, *10*